

MAINHEAD Capability Ownership Review

Does assigning capabilities to a MAINHEAD provide real operational value in the current resolver?
Assessment only — no code changes.

HEADLINE — MAINHEAD capability assignment is currently a grant, not a filter. Capabilities placed on a MAINHEAD are silently promoted into the effective capability set of every user with access to that MAINHEAD, via the additive union in the resolver (`users.service.ts:627–741`). This creates governance widening that conflicts with the workspace-gate semantics of the pilot. Recommendation: Restrict — keep MAINHEAD capability assignment for Asset Domains as a scope label, drop it from Workspace and Governance categories, and stop the resolver from promoting MAINHEAD capabilities to user authority. Full reasoning in §4.

1. Current inheritance chain — what the resolver actually does

“Inheritance” is the operative word in the brief, but the implementation is not inheritance — it is an **additive union** across five capability tables. Source: `UserService.getEffectiveCapabilitiesForUser` (`apps/api/src/users/users.service.ts`, lines 485–804).

1.1 Step 1 — Collect scope IDs

For the target user the resolver builds four sets:

- **teamIds** — every team the user is a member of (`userData.teamMemberships`).
- **directMainheadIds** — every MAINHEAD the user has access to, via the union of: (a) `UserMainheadAccess`, (b) MAINHEADs derived from each team's `team.mainheadId`, (c) MAINHEADs derived from `UserOperationalRegionAccess` (every MAINHEAD whose `operationalRegionId` is one of the user's region IDs), (d) the legacy `userData.mainheadId` column.
- **branchIds** — direct `userData.branchId`, plus branches derived from each team's `team.branchId`, plus branches derived from each MAINHEAD's `branchId`.
- **organizationIds** — direct `userData.organizationId`, plus orgs from `organizationMemberships`, plus orgs derived from each branch's `organizationId`, plus `team.organizationId`.

ADMIN role short-circuits: an ADMIN user's effective set is every active Capability row, tagged scope: 'ADMIN'.

1.2 Step 2 — Fetch five capability sets in parallel

Five concurrent Prisma queries:

```
userCapability.findMany    where userId, active
teamCapability.findMany    where teamId in teamIds, active
mainheadCapability.findMany where mainheadId in directMainheadIds, active
branchCapability.findMany  where branchId in branchIds, active
organizationCapabilityAssignment.findMany
                             where organizationId in organizationIds, active
```

Each row carries a capability reference and a scope label.

1.3 Step 3 — Merge into one set, additively

All five result sets are flattened into a single Map<capabilityId, EffectiveCapability>. Each contribution appends its source onto the existing entry; no contribution can **remove** a capability another source granted. There is no intersection step, no precedence rule, and no filter step.

1.4 Step 4 — Consumers

Consumer	What it does with the set
Mobile workspace resolver (apps/mobile/src/operationalWorkspaces)	Gates Inspection and Maintenance workspaces by presence of INSPECTION / MAINTENANCE in the effective set. ADMIN users see both.
GET /users/me/capabilities + GET /users/:id/capabilities	Displayed as-is for admin transparency. Sources are returned so the admin can see <i>why</i> a user has each capability.
Templates / Defects / Inspections / QA / Asset Types	None. Confirmed by grep — no template resolver, defect governance, inspection workflow, or asset-type filter reads any of the five *Capability tables.

Net effect: the only operational behaviour that depends on MAINHEAD capability assignments today is mobile workspace gating, mediated through additive merging.

2. Where MAINHEAD capabilities provide real value

Scenario	Why it works
"KL Timur performs SAVR and PENCAWANG; KL Barat additionally performs CABLE_BRIDGE."	Treats MAINHEAD as an operational-area label . Capabilities here describe what kinds of asset work happen in that geography, which matches the V10 definition of MAINHEAD as the operational area.
Contractor team is reassigned from KL Barat to KL Timur partway through pilot.	If the team had no per-team capability set, MAINHEAD capabilities provide "local scope" — the team automatically picks up the work types relevant to the new area. Avoids hand-editing team capabilities each rotation.
Operations management asks: "what work types is each MAINHEAD authorised to perform?"	MAINHEAD capability assignments make this answerable in one place, instead of aggregating across all teams that touch the area.
Reporting: defect rates by asset domain per MAINHEAD.	MAINHEAD capabilities act as a denominator filter ("areas authorised for FEEDER_PILLAR") for area-level KPIs. Useful for the future Reporting Engine in V10 §50.
Pilot setup: define a single MAINHEAD with SAVR; technicians inherit the scope.	Reduces admin steps during pilot ramp-up. One MAINHEAD edit replaces N team edits.

These scenarios all share one property: the capability is **describing what happens at the area**, not **granting individuals authority**. The model serves them well only when used as a label.

3. Where MAINHEAD capabilities create governance confusion

Scenario	What goes wrong
SAVR-only technician has access to KL Timur, which also lists CABLE_BRIDGE.	The resolver adds CABLE_BRIDGE to the technician's effective set. Workspace gating only checks INSPECTION/MAINTENANCE, but any future asset-type or template filter that reads effective capabilities would surface CABLE_BRIDGE assets to a SAVR-only technician. Silent privilege widening.
User has Region access to Klang Valley. Klang Valley has 6 MAINHEADs. One of them is assigned MAINTENANCE.	Resolver promotes all 6 MAINHEADs' capabilities into the user's effective set. Mobile workspace resolver now opens the Maintenance workspace for the user even though they are an inspection technician. Multi-MAINHEAD union compounds the widening.
Admin assigns QA_VALIDATION to a MAINHEAD by mistake.	Per V8 §15, QA authority is cross-MAINHEAD and ASCURA-owned. MAINHEAD-level QA_VALIDATION has no defined semantics. Today the assignment is accepted, gets merged into every user's effective set for that area, and produces a QA_VALIDATION tag on technicians who should never have it. No validation, no error.
Workspace gates were just established by the pilot (INSPECTION, MAINTENANCE).	Granting INSPECTION to a MAINHEAD gives every user-with-access the Inspection workspace. That's a user-authority grant pretending to be area metadata. Workspace gates are bypassable via MAINHEAD assignment.
Auditor asks: "why does user X have FEEDER_PILLAR?"	The sources array tags one row as scope: 'MAINHEAD', but you must walk back through UserMainheadAccess + region inheritance + team's team.mainheadId + legacy mainheadId to know which path put it there. Audit cost.
Per V6 §12: contractors must remain decoupled from MAINHEAD ownership.	When you place a capability on a MAINHEAD, you're encoding "this area performs X kind of work" — which is contractor-shaped behaviour, on an area entity that's supposed to be contractor-neutral. Conflicts with the contractor-independence principle.
Removing a MAINHEAD capability does not remove user authority.	Because the resolver is purely additive, the same capability often also flows from Team or Org. Untoggling the MAINHEAD row produces no observable change in the effective set if any other source contributes the same capability. Cleanup is unreliable.

4. Recommendation

Restrict MAINHEAD capability assignment. Do not keep as-is. Do not remove outright.

4.1 Why not Keep

- Today's additive resolver makes MAINHEAD capability a grant, not a filter. Combined with multi-MAINHEAD union and region inheritance, that produces unbounded privilege widening (§3).
- Workspace gates established by the pilot can be bypassed by any admin who ticks INSPECTION or MAINTENANCE on a MAINHEAD.
- Mis-assignment of governance codes (QA_VALIDATION, REPORTING) at the MAINHEAD level is silently accepted today.

4.2 Why not Remove (yet)

- Asset-domain scoping at the area level is a **genuine** use case (§2 row 1, row 4). Removing the table loses that signal entirely.
- Removal requires a Prisma migration on a live pilot DB, plus deletion of MainheadCapability writes/reads across `enterprise.service.ts`, `users.service.ts`, and the MAINHEAD admin form. Larger blast radius than the pilot needs right now.
- The capabilities catalogue and the Capability Picker (Items 1 and 4 of the Pilot UX Cleanup) just stabilised. A bigger structural change would invalidate the rollout we just signed off.

4.3 What Restrict means concretely

- **UI restriction.** In the MAINHEAD admin modal (`enterprise-list-client.tsx`, MAINHEAD branch of the form), hide the Workspace Access and Governance & Reporting fieldsets. Show only the Asset Domains fieldset. The `capability-groups.ts` helper already returns groups by key, so a per-context allow-list is trivial.
- **Server-side validation.** In `syncMainheadCapabilities` (`enterprise.service.ts:1969`), reject any `capabilityId` whose code is not in `ASSET_DOMAIN_CODES`. Returns 400 with a clear error. Stops API consumers from bypassing the UI.
- **Resolver change.** In `getEffectiveCapabilitiesForUser`, exclude `mainheadCapabilities` from the merged effective set used for workspace gating, OR scope it so its contributions only flow when the user already has a matching User/Team grant. Keep the source tag for transparency ("MAINHEAD KL Timur performs SAVR"), but stop the silent authority promotion.
- **Backfill / cleanup.** Run a one-shot script to identify any existing `MainheadCapability` rows whose code is not in the Asset Domains allow-list, and surface them in a report for ops to review. Do not delete automatically. Mirrors the legacy-capability handling pattern.
- **No mobile change.** Mobile reads the resolver output. If the resolver stops promoting MAINHEAD-sourced workspace capabilities, mobile behaviour tightens automatically — and matches the workspace-gate semantics the pilot just shipped.

4.4 Effect after Restrict

Concern	Before	After Restrict
INSPECTION/MAINTENANCE assignable on MAINHEAD	Yes — silently grants workspace	No — hidden in UI + rejected by server
QA_VALIDATION/REPORTING assignable on MAINHEAD	Yes — no effect except misleading audit	No — hidden in UI + rejected by server
Asset-domain scope on MAINHEAD	Yes — but contributes to user authority	Yes — but does not contribute to user authority. Acts as label only.
Cross-MAINHEAD privilege widening via region access	Yes — region access pulls in every MAINHEAD's capabilities	No — region access still grants area visibility, but not capability grants from each area
Audit clarity	Five-way union with overlapping sources; hard to remove a single source	User authority comes from User + Team + Org/Branch only. MAINHEAD remains a labelling layer.

4.5 Out of scope for this assessment

- The same additive-union concern applies to **Organization** and **Branch** capability assignment, though with smaller blast radius (Org/Branch don't have the region-inheritance multiplier). Flag for a future review after the MAINHEAD restriction settles.
- `UserOperationalRegionAccess` currently inflates `directMainheadIds` with every MAINHEAD in the region. Restriction here does not change that, but combined with the Effective MAINHEAD Access preview shipped in the Pilot UX Cleanup, admins can at least see the expansion before saving.
- Long-term: introduce an explicit `Capability.kind` column (WORKSPACE / GOVERNANCE / ASSET_DOMAIN) so server-side validators don't depend on the client-side allow-list constant.

5. Bottom line

MAINHEAD capability assignment has a real and useful semantic — describing what kinds of asset work happen in a given operational area. It also has, under the current resolver, an unintended semantic: every capability placed there is silently promoted into every accessing user's authority set, including workspace gates and governance roles. Restricting the assignable scope to Asset Domains, plus stopping the resolver from promoting MAINHEAD-sourced rows into user authority, keeps the useful signal and eliminates the governance leak. Remove is premature; Keep is unsafe; **Restrict** is the pilot-grade path.